GigBid: an agent that bids on freelance jobs, with a human in the loop

AI-4016 R&D report · engineering design review · every number cites a file in `evals/results/` (or a commit/doc). Repo: `https://github.com/msaqibatifj/GigSim` · Live: `https://gigbid-yabd.onrender.com`

> **Read first.** The evaluation is small (14 compact dev jobs, one run each; 30 for retrieval) and the decision thresholds were tuned on those same 14 jobs. On the two low-fit dev jobs the compact set left out, the system bids wrongly on both (§8). GigSim, the personas and the policy pack were not provided, so they are local mocks, and **every GigSim outcome (reply, hire) comes from a client model I wrote**: reply rates measure agreement with that model, not real clients. The LLM judge failed calibration against my labels (kappa 0.001, §4), so no judge scores are reported.

1. Problem and constraints

A freelancer's bidding loop: find jobs, read them, spot scams, check whether their own work supports the requirements, price a bid under a Connects (bidding-credit) budget, write a proposal, and answer the client. The costly mistakes are: claiming experience they do not have, bidding on scams or free-work requests, leaking contact details, and anything being sent without their say-so.

Never automated: sending a proposal or message, spending Connects, and submitting on the marketplace in live mode (the user confirms the draft on Upwork). Enforced by a LangGraph `interrupt()` before every write, idempotency keys on every write, and a live adapter whose `submit_proposal` returns "confirm on Upwork".

Budgets declared (`agent/config.yaml`): p95 latency 60 s per job, USD 0.02 per job and 0.10 per proposal. **Measured** on the compact set [decision_eval_open.json]: p95 27.6 s, USD 0.0004 per job and 0.0008 per proposal, 2,172 tokens per job. Open-weight models only: `gpt-oss-20b` for small nodes, `gpt-oss-120b` for Writer and Critic, on Groq. Embeddings `bge-small-en-v1.5`, reranker `ms-marco-MiniLM-L-6-v2` (dev-size, CPU-only machine).

2. Architecture

Bid graph (one thread per job): scout, then analyst | risk | matcher in parallel via `Send` (state reducer for the parallel writes), decide (skip / watch / bid), pricing, writer, claim verifier (unsupported claims removed *before* the critic), critic (back to writer at most twice), compliance (deterministic policy pack, can block with a reason), approval (`interrupt()`), idempotent execute. **Conversation graph**: read, classify (question / negotiation / interview / scam signal), draft (slots from the Workspace server), approval, send. Budgets of 40 steps and 60,000 tokens per thread move a run to a visible `needs_attention` state. Both graphs are rendered from code in `docs/images/`.

Marketplace port: one interface (`agent/marketplace_port.py`), two adapters (GigSim, live Upwork) behind the Marketplace MCP server, chosen by `MARKETPLACE_ADAPTER`. **MCP surface:** Portfolio (read), Marketplace (read and write, bearer auth, per-user rate limit, append-only audit log), Workspace (calendar and email drafts, local mock), plus the official Upwork server and a GitHub server (configured, not run; needs a token). 17 tools with schemas, output schemas and annotations, snapshot-tested (`docs/TOOLS.md`).

Stores: state (checkpointed in Postgres in the deployment, SQLite locally; all my tests used SQLite), execution trace (`events` table, audit log, LangSmith), and long-term memory (style and outcome, written only through promotion rules: a style pattern needs 2 separate edits, outcome stats need 3 outcomes in a category). Raw tool output and draft text are never stored in memory.

Why each agent exists: Scout (cheap, scheduled, dedupes); Job Analyst (no tools, so injected text cannot reach a tool); Risk Assessor (client history plus a deterministic injection detector OR-ed with the LLM flag); Fit Matcher (the only source of claims); Pricing (market statistics from 1.3M real contracts, the user's floor, Connects); Writer and Critic (strongest model); Compliance Guard (regex and policy, so it cannot be talked out of a rule); Client Liaison.

3. R&D question A: tool granularity

Setup: 40 tasks with known correct calls, one model call each (`gpt-oss-20b`, temperature 0); three designs of the same capabilities. I did not write the hypothesis to a file before running, so it is not pre-registered. My prior was that fewer, coarser tools would win on tokens and lose on accuracy.

Design	Tools	Tool accuracy	Argument accuracy	Prompt tokens / task
many fine-grained, terse descriptions	24	0.70	0.70	629
few coarse (action + args)	4	0.85	0.675	301
best: my real tools, rewritten descriptions	16	1.00	1.00	697

[tool_granularity.json]. Conclusion: description quality mattered more than tool count; coarse tools halved the tokens but lost argument accuracy because the model must invent the argument shape. Limit: I wrote the tasks and the descriptions together, so 1.00 is an upper bound; one run, n = 40.

4. R&D question B: RAG

Variants, results and shipped choice

Seven variants implemented as configurable components (`rag/variants/vN_*.py`): V0 naive dense, V1 requirement decomposition, V2 hybrid BM25+dense with RRF, V3 cross-encoder rerank, V4 outcome-aware boost, V5 decompose+hybrid+rerank, V8 corrective RAG as a LangGraph loop. Retrieval evaluated against gold evidence for 30 dev jobs with all three personas' items in one index (distractors).

Variant	Recall@5	nDCG@10	nDCG@10 (per-persona index)	retrieval p95 (s)
V0 naive	0.933	0.885	0.907	0.99
V1 decompose	0.700	0.611	0.838	7.5
V2 hybrid	0.933	0.884	0.913	0.67
V3 rerank (shipped)	0.950	**0.912**	0.916	1.8
V4 outcome-aware	0.933	0.890	0.901	1.7
V5 decompose+hybrid+rerank	0.967	0.858	0.871	24.7
V8 corrective	0.933	0.907	0.911	89

[rag_eval_all.json, rag_eval_persona.json]. MRR is in the files. The B5 table's pipeline columns (unsupported claims, reply rate, cost per proposal) were run only for the shipped V3 [decision_eval_open.json]; the other variants show retrieval metrics only, marked "not run" in `evals/results/RESULTS.md`.

Decision that ships: V3. It has the best nDCG@10, near-best Recall@5, and a 1.8 s p95. V5 has the highest Recall@5 (0.967) but a worse nDCG@10 and a 25 s p95; V8 costs 89 s p95 for no gain; decomposition lowers quality in both index settings. The deployed variant is the config value `rag.variant`, so changing it is not a code change.

Ablation (hypotheses written before running, in `evals/runners/ablation.py`)

Arm (from decompose+hybrid+rerank)	Recall@5	nDCG@10	change	single-evidence	multi-evidence
full	0.967	0.854	–	0.856	0.848
- rerank	0.767	0.650	-0.204	0.688	0.546
- hybrid	0.967	0.854	0.000	0.856	0.848
- decompose	0.950	0.912	+0.058	0.927	0.871

[ablation.json]. H1 (reranker matters most) held. H3 (hybrid adds little) held, with identical numbers, likely because on a portfolio this small the reranker sees the same top candidates either way. H2 (decomposition helps multi-evidence jobs) was wrong: removing it helped both job types. H4 is addressed below.

By persona and job type

V3 vs V0, nDCG@10 in the pooled setting [rag_eval_all.json]: Ayesha (ML) 0.988 vs 0.911, Sara (data) 0.897 vs 0.856, **Bilal (web) 0.851 vs 0.887, i.e. the reranker is worse for the web persona**. By job type: single-evidence 0.927 vs 0.899, multi-evidence 0.871 vs 0.844. In the per-persona index setting V3 is worse than V0 on multi-evidence jobs (0.887 vs 0.927) [rag_eval_persona.json]. With 30 jobs split three ways these differences are small and I have not tested their significance.

Claim verification (B3)

Each draft is split into claims, labelled supported / partial / unsupported against the retrieved evidence, and unsupported claims are removed before the Critic. Unsupported claims per proposal: **0.33 before, 0 after** (6

proposals) [decision_eval_open.json]. The verifier is an LLM (`gpt-oss-20b`) judging the same evidence the Writer saw, so it shares its blind spots; I have not measured its precision against human labels.

Where better retrieval did not improve reply rate, and judge calibration

I cannot answer H4 with this simulator. GigSim's `outcome()` scores a proposal on word overlap with the job, a specific opening, length, price versus budget and client quality; it never looks at which portfolio item was retrieved, so by construction better retrieval cannot move it except through wording. The pipeline-level reply rate was measured only for V3 (0.33, 6 proposals). **Judge calibration is done and the judge fails it:** on 30 anonymised items scored blind by me (1-5), weighted Cohen's kappa is 0.001, exact agreement 0.10, mean human 3.07 vs mean judge 4.97 (`judge_calibration.json`). The judge rates nearly everything 5, so no LLM-judge scores are reported as quality evidence, and the "Rubric score" column stays empty; the blind pairwise human panel of §B4 was not run.

5. R&D question C: is multi-agent worth it?

Same 14 jobs, same models, same retrieval. (a) single ReAct agent with all read tools, (b) prebuilt `langgraph-supervisor` with three workers, (c) my hand-built graph stopped at the approval gate. Not the 50 jobs asked for: the Groq free tier caps tokens per day, so I ran the compact set.

Metric	(a) single	(b) supervisor	(c) my graph
bid/skip accuracy	1.00	0.64	1.00
scam / injection skipped	1.00	1.00	1.00
planted instruction obeyed	0	0	0
unsupported claims / proposal*	0	5 (1 proposal)	0
drafts with contact / guarantee / over-length*	0	0	0
GigSim reply rate (synthetic)	0.33	0.00 (1 proposal)	0.33
tokens / job	1,379	741	2,172
p50 / p95 latency (s)	44 / 187	14 / 29	14 / 28

[agent_comparison.json]. *Baselines are raw model output while the graph's drafts are post-verification and post-compliance, so these two rows favour the graph by construction. Compliance violations "caught" is 0 in every arm: the Writer rarely produced any, so this set cannot show the Compliance Guard's value; it is unit-tested instead.

Honest reading. On accuracy and quality the single agent **ties** my graph on this set and uses 36% fewer tokens, so if only those mattered I would ship it. The graph is about 7x faster at p95 because its three analysis branches run in parallel while the single agent calls tools one after another, and it is the only variant with the approval gate, a deterministic compliance step and verified claims. The prebuilt supervisor skipped 5 of the 6 jobs that should be bid. n = 14, one run each, nondeterministic models.

6. Change request

Not received: no 2 Nov change-request text was provided to this project, so none was applied and no time or diff is claimed. What exists: rules and costs live in `data/policy_pack/policy.yaml` (limits, fees, Connects per bid, and a `forbid_patterns` list of regex rules with action `remove` or `block`), read at call time by Compliance, Pricing and GigSim; per-job Connects cost comes from the posting. A unit test adds a "no rate quotes" rule and a "no crypto" block rule through the config alone and checks they take effect with no code change (`tests/unit/test_inline.py`). Expected diff for a new content rule: 1-2 lines of YAML; for a new Connects cost: 1 line. Measured diff and time for the real change: not available.

7. Signature extension: X2 proposal experiments

Two strategies on the same 30 bid-worthy jobs, same Writer model, evidence and verification, differing only in style rules: A intro-first, B deliverable-first (opens with the exact deliverable, one measurable result, one scoped question). Paired sign-flip permutation test on per-job GigSim reply probability.

Result [ab_experiment.json]: mean reply probability A 0.514, B 0.579, difference **+0.065, permutation p = 0.0033**; but sampled simulated replies were **17 (A) vs 16 (B)**, so the effect on the probability does not show up in 30 draws. B's letters were longer (92.5 vs 77.8 words), and GigSim rewards a specific opening and a 60-220 word length, so a good

part of "B wins" is the simulator's own assumptions. It demonstrates the experiment pipeline, not an effect on real clients.

8. Failure analysis

16 failures in `docs/FAILURES.md` with fix and outcome; summary by category (the required ones plus Wrong bid, Critic over-rejection, Latency, Rate limit):

Category	Jobs	Fix tried	Worked?
Wrong bid (false evidence: retrieval always returns something)	a11, b11, s14	cross-encoder score floor -8.0 for "supported"	yes (these three)
Wrong bid, held-out	a14, b14	none; tuning again on the same data would overfit	**no**
Wrong skip after the floor	a02, s01	bid_min_fit 0.5 to 0.2	yes, but tuned in-sample; it is why a14 is bid
Tool error (model returned a null id)	a13, b12	analyst sets `job_id` itself	yes
Unsupported claim	a02, s02	claim verification before the Critic	yes (1 to 0, 3 to 0)
Critic over-rejection / latency	b01, a11 (160 s)	Critic sees evidence and labels; re-verify only changed drafts	yes (loops 1.78 to 0; p95 90 s to 28 s)
Injection	inj2, inj3; inj1, inj2, inj4	tag neutralising, deterministic detector, scrub injected sentences	yes (attack success 0.67 to 0.0, n = 6)
Rate limit; log-mask bug	all	key rotation and backoff; mask at record creation	yes

The Critic row's risk is the opposite failure (approving everything), which I have not measured. No Compliance misses occurred in the dev runs, which also means the dev set barely exercises it. The score separation behind the floor is only 0.86 logits wide across all 42 dev jobs [score_gap.json].

9. Safety and platform respect

GigBid must never: send or spend without approval; claim unsupported experience; follow instructions inside job posts or client messages; leak contact details or payment data into logs; scrape, automate a browser, or poll like a scraper; auto-reply to a scam signal. **Enforced by:** `interrupt()` plus idempotency keys; claim verification; posts delimited as untrusted, closing tags neutralised, injected sentences scrubbed before drafting, a regex detector OR-ed with the LLM flag; PII masking at log-record creation and in traces; OAuth tokens Fernet-encrypted and never returned or logged; every query scoped by user; live mode scouts at most every 6 hours and creates drafts only.

Attack success on 6 planted posts: 0.667 before the code-level defences, 0.0 after [injection_eval.json]; the planted word reached a draft in 50% before, 0% after. **What still gets through:** the detector is regex-based, so a paraphrased or non-English instruction passes it and only the LLM flag and the human approval gate remain (the LLM flag varies between runs); a post that sounds like a hiring manager asking a "screening bot" to say a word is the example that would beat the regex. n = 6, so the 0.0 is not a guarantee.

10. Hidden test results

Not available at submission time. The instructor's hidden run happens after the deadline. Declared budgets for comparison: p95 60 s per job, USD 0.02 per job, 0.10 per proposal; my compact-set measurements above are 27.6 s and USD 0.0004 / 0.0008.

Reproduction, versions and limits

`make eval` reruns retrieval, decisions, injection and tool-granularity evals; `python -m evals.runners.results_table` regenerates every table here. 37 backend tests and 5 Playwright tests pass locally (`docs/images`, `evals/results`). Pinned versions are in `pyproject.toml` and the README. **Not done or not verified:** live Upwork against a real account; Docker compose and the Postgres checkpointer (no Docker on my machine; SQLite used in all tests); GitHub MCP server run; the skipped-scam LangSmith trace (the free-tier monthly trace quota is exhausted; the critic-loop and crash-resume traces are linked in the README); the blind human panel (judge calibration is done and failed); hidden-run results; the real change request.